

SO-Bootstrap

Para empezar a entender como funciona el bootstrap de un ordenador es importante entender como funcionan las interrupciones de dispositivos de E/S para teclados, mandos de juegos o cámaras.

Interrupciones

Cuando no se está pulsando una tecla, la CPU está ocupada procesando otras tareas, como ejecutar una aplicación. Los dispositivos de entrada están en estado de reposo.

- En el momento que se pulsa una tecla, el controlador de hardware del dispositivo detecta un cambio eléctrico.
- El dispositivo envía una señal eléctrica a través de una línea física dedicada llamada **IRQ (Interrupt Request Line)** hacia el Controlador de Interrupciones. Esencialmente, es un "toque al hombre" electrónico a la CPU.

En ese momento, ocurre lo siguiente:

1. **Interrupción del flujo:** La CPU termina la instrucción que estaba ejecutando en ese microsegundo, guarda el *Program Counter* y el *Status Register* en la pila y se prepara para atender la interrupción.
2. **Identificación:** La CPU consulta al **Controlador de Interrupciones** para saber qué dispositivo es el que está reclamando atención. Cada dispositivo tiene un número de identificación único (vector de interrupción).
3. **Ejecución de la ISR:** La CPU salta a una dirección de memoria específica donde reside el **Driver (manejador)** de ese dispositivo.
 - *Si es un teclado:* La ISR lee el código de la tecla pulsada del registro del controlador de teclado y la coloca en un búfer de memoria.
 - *Si es una cámara:* La ISR gestiona la transferencia de un bloque de datos (fotogramas) desde el búfer de la cámara hacia la RAM (a menudo usando **DMA** para no cargar a la CPU con el peso de los píxeles).

Una vez que el Driver ha procesado el dato (por ejemplo, ha registrado que se presionó la letra "A"), ejecuta una instrucción especial de retorno. La CPU recupera el estado que guardó al principio y continúa con la aplicación de usuario exactamente donde se quedó.

¿Como cambia la interacciones según el dispositivo?

Dispositivo	Comportamiento de la Interrupción
Teclado	Genera interrupciones de la baja tasa de datos . Cada pulsación es un evento independiente. Es muy eficiente porque la CPU solo trabaja cuando tú decides pulsar una tecla.
Mando de juegos	Genera una ráfaga constante de interrupción mientras la palanca está en movimiento (polling por interrupción). Son eventos muy rápidos para evitar latencia en los juegos.
Camara (Webcam)	No usa interrupciones para cada pixel . Sería catastrófico para el rendimiento. La cámara usa el DMA (Acceso Directo a Memoria) : el hardware llena una sección e la RAM con la imagen y, al terminar el bloque, envía <i>una sola interrupción</i> a la CPU diciendo: "El fotograma está listo en la memoria, ya se puede procesar".

¿Quién maneja la máquina si el SO aún no ha cargado?

La respuesta corta es que la **BIOS (o el moderno UEFI)** es, en si mismo, un pequeño sistema operativo muy básico diseñado para preparar el terreno para que el sistema operativo "real" (Windows, Linux, etc.) pueda tomar el control.

El entorno Previo al SO - BIOS

Cuando se enciende el computador, el procesador (CPU) está en un estado muy básico. No hay drivers, no hay gestión de archivos, solo hay un código quemado en un chip de la placa base llamado **Firmware**.

- La BIOS **no usa un microkernel**. Es un entorno de **ejecución lineal** o basada en eventos muy simples. En las BIOS antiguas (Legacy), era código puro en ensamblador que ejecutaba tareas paso a paso. En las modernas (UEFI), es un entorno mucho más complejo que permite cargar drivers básicos, gestionar redes y hasta tener interfaces gráficas.

- Incluso antes del SO, el hardware ya puede generar interrupciones. La BIOS carga su propia "Tabla de Vectores de Interrupción". Es como un índice: "Si ocurre la interrupción X (tecla pulsada), ejecuta esta pequeña subrutina que está en la dirección Y de la memoria".

Para que se pueda utilizar el teclado o el mouse en la configuración de la BIOS, el firmware tiene **sus propios drivers extremadamente limitados**:

1. **Drivers de Firmware**: La BIOS tiene drivers genéricos para teclado (protocolo PS/2 o emulación USB HID) y para el mouse
2. **Polling vs Interrupciones**: A diferencia del SO, muchas BIOS antiguas funcionaban mediante *polling*, pregunta en cada momento si se ha pulsado alguna tecla. Las BIOS modernas (UEFI) ya utilizan un sistema de interrupciones similar al del SO para no bloquear el procesador mientras esperas a que el usuario mueva el mouse.
3. **El "Modo de Gestión del Sistema" (SMM)**: Existe un modo especial del procesador llamado **System Management Mode**. Es un modo de ejecución de altísima prioridad, invisible para el SO que tú conoces, que permite que el firmware del hardware mantenga el control total sin importar qué esté pasando en la capa superior.

La transición

El proceso es una cadena de relevos:

1. **POST (Power-On Self-Test)**: El firmware comprueba que la RAM y la hardware funcionen.
2. **Entorno Interactivo (BIOS/UEFI)**: Al pulsar la tecla de configuración (ej. **F2**), el firmware detiene la carga. Aquí es donde usas el mouse. La BIOS tiene una pequeña rutina gráfica que dibuja el cursor y lee las coordenadas del mouse, traduciéndolas a clics sobre sus propios menús.
3. **Bootloader (El puente)**: Cuando sales de la BIOS y eliges arrancar el SO, la BIOS busca un archivo especial en tu disco duro (el gestor de arranque o *bootloader*, como GRUB o el gestor de Windows).
4. **Carga del Kernel**: El *bootloader* es el último programa que ejecuta el firmware. Este programa le da Kernel del SO una lista de "todo lo que encontré" (qué discos hay, cuánta RAM existe) y **le transfiere el control de las tablas de interrupciones**. A partir de aquí, el firmware "se retira" y el Kernel del SO toma el mando de todas las interrupciones.

Resumen

No es un sistema operativo completo, pero tiene todas las piezas básicas:

- **Driver**: Un programa mínimo para entender el hardware
- **Gestor de Interrupciones**: Un mapa básico para saber qué hacer cuando pulsas una tecla.
- **Interfaz**: Un código simple que dibuja píxeles en pantalla (en BIOS modernas).

Boot Sequence

Cuando el computador se enciende, la BIOS/UEFI realiza un inventario del hardware. Entre sus configuraciones, tiene una **lista de prioridad de arranque**. El firmware busca en el sector 0 (el primer bloque de datos) de cada dispositivo un pequeño código especial llamado **MBR (Master Boot Record)** o, en sistemas modernos, una **partición EFI**.

- **En el CD**: La BIOS busca un estándar llamado **El Torito**. Este formato le dice a la BIOS que el CD tiene un sector de arranque que carga los datos del sector en la memoria RAM y salta a esa dirección.
- **En el USB**: La BIOS trata el USB como si fuera un disco duro. Busca el MBR o la partición EFI. Si el USB está formateado correctamente (con un sector de arranque válido), el firmware lo reconoce como un "dispositivo arrancable".

Cuando se selecciona arrancar desde USB/CD y ocurre lo siguiente:

1. **Carga del "Bootloader"**: La BIOS/UEFI copia esos primeros sectores del CD o USB directamente a una zona de la memoria RAM.
2. **Transferencia de control**: La BIOS le dice al procesador que deje de ejecutar el código actual, mueva el PC a la dirección de memoria donde está el código del USB.
3. **Ejecución**: En ese momento, el programa tiene el control total de la máquina. No hay SO.

¿Cómo maneja pantalla, teclado y datos?

Como no hay sistema operativo que le dé drivers al dispositivo de arranque, el programa **debe traer sus propios drivers básicos dentro de su paquete de arranque**.

- Si es programa es muy pequeño (tipo DOS) utiliza las "Interrupciones de BIOS" (INT 10h para video, INT 16h para teclado). El programa no sabe cómo manejar la tarjeta gráfica, simplemente pide a la BIOS que lo haga por él.
- Si el programa es complejo (como una herramienta de clonación o un instalador moderno) el programa trae su propio "mini-sistema". Suele ser una versión ultra reducida de un kernel (como un Linux embebido o un Windows PE - Preinstallation Environment). Estos programas cargan sus propios drivers en la RAM para poder manejar mouse USB modernos, redes o pantallas de alta resolución.